

Atividade 02

Vence amanhã às 23:59•Vários envios permitidos•

Fecha amanhã às 23:59

Instruções

O **Bubble Sort** é um algoritmo de ordenação simples que funciona comparando pares de elementos adjacentes e trocando-os de posição quando estão fora de ordem.

Esse processo é repetido até que a lista esteja completamente ordenada.

Animação interativa para exemplificar o

método: <https://yongdanielliang.github.io/animation/web/BubbleSortNew.html>

Como funciona:

1. Percorre a lista comparando cada par de elementos.
2. Se o elemento atual for maior que o próximo, troca-os de posição.
3. Repete as passagens até não haver mais trocas.

EXEMPLO: (Lista: [6, 3, 8, 5, 2])

Primeira passagem:

- Compara 6 e 3 → troca → [3, 6, 8, 5, 2]
- Compara 6 e 8 → não troca → [3, 6, 8, 5, 2]
- Compara 8 e 5 → troca → [3, 6, 5, 8, 2]
- Compara 8 e 2 → troca → [3, 6, 5, 2, 8]

Maior elemento (8) foi para o final.

Segunda passagem:

- Compara 3 e 6 → não troca → [3, 6, 5, 2, 8]
- Compara 6 e 5 → troca → [3, 5, 6, 2, 8]
- Compara 6 e 2 → troca → [3, 5, 2, 6, 8]

Segundo maior (6) ficou na posição correta.

Terceira passagem:

- Compara 3 e 5 → não troca → [3, 5, 2, 6, 8]
- Compara 5 e 2 → troca → [3, 2, 5, 6, 8]

Quarta passagem:

- Compara 3 e 2 → troca → [2, 3, 5, 6, 8]
- Nenhuma troca necessária depois → lista ordenada.

TAREFA:

1. Implemente o *Bubble Sort* em Python.
2. Conte o número de comparações e trocas realizadas durante a ordenação.
3. **Teste os três cenários:**
 - **Melhor caso:** Lista já ordenada (ex.: [1, 2, 3, 4, 5]).
 - **Médio caso:** Lista em ordem aleatória (ex.: [3, 5, 1, 4, 2]).
 - **Pior caso:** Lista em ordem inversa (ex.: [5, 4, 3, 2, 1]).
4. **Compare os resultados** e explique por que o número de operações varia.
5. Compare também os 3 cenários com o ***algoritmo Selection Sort***.

Entrega:

- Enviar um único arquivo em pdf com os códigos-fonte e as análises.
- Pode ser em até trios.